TAREA. IMPLEMENTACIÓN DE LA ESTRUCTURA COMPLETA DE UNA BASE DE DATOS (DDL)

INTEGRANTES:

SONIA MARIA GOMEZ BOLIVAR
CAMILA ANDREA GOMEZ ORTIZ

PROGRAMA:

TECNOLOGÍA EN DESARROLLO DE

SOFTWARE G100 (SD1006) BASES

DE DATOS I

DOCENTE:

ORALIA CORTES
GRAJALES

INSTITUCIÓN UNIVERSITARIA

PASCUAL BRAVO MEDELLÍN,

COLOMBIA

2026-2

INTRODUCCION

El presente trabajo tiene como propósito implementar el modelo lógico de la Red Social Estudiantil Pascualina en un Sistema Gestor de Base de Datos (SGBD) real, aplicando las reglas y estándares del lenguaje de definición de datos (DDL). La tarea se enmarca en la necesidad de diseñar una plataforma que facilite la interacción entre estudiantes, permitiendo la creación de perfiles, la organización de grupos de estudio, la publicación de contenidos y la programación de eventos académicos y sociales.

La importancia de este ejercicio radica en que, además de fortalecer las competencias técnicas en el manejo de bases de datos, fomenta el uso de buenas prácticas de modelado y normalización, garantizando la integridad y coherencia de la información. Asimismo, el proceso de implementación en PostgreSQL permite a los estudiantes adquirir experiencia práctica en un entorno profesional ampliamente utilizado en la industria del software.

Estructura de Avance del Proyecto

1. COMPARACIÓN DE SGBD PARA LA RED SOCIAL PASCUALINA

Criterio	PostgreSQL **(Recomendado elegido)**	**MySQL**
Tipo de datos para textos	Excelente rendimiento con test (comentarios y biografías) sin penalizar almacenamiento.	Buen manejo de `TEXT`, pero con ciertas limitaciones de indexación en textos muy masivos.
Relaciones y restricciones	Estricto y robusto. Ideal para asegurar que las conexiones/mentorías mutuas no fallen.	Flexible, aunque históricamente menos estricto con la verificación implícita de restricciones.
Escalabilidad	Diseñado para consultas complejas (ej: saber qué intereses comparte un estudiante con sus seguidores).	Muy rápido en lecturas simples, pero puede ralentizarse con múltiples uniones (JOIN) cruzadas.
Concurrencia	Excelente manejo de múltiples usuarios interactuando a la vez (likes, comentarios simultáneos).	Utiliza bloqueos de filas eficientes, pero puede sufrir con alta escritura concurrente en la misma tabla.

Justificación de la elección de sistema gestor de base de datos SGBD

Como se trata de una red social estudiantil, el sistema necesita realizar múltiples uniones (JOIN) a la vez para conectar perfiles con interés seguidores, grupos de estudio mentorías PostgreSQL tiene un optimizador de consultas avanzado ideal para este tipo de carga relacional masiva.
Además, la plataforma maneja una cantidad enorme de texto libre como por ejemplo en biografía, publicaciones, descripción de eventos y comentarios. PostgreSQL administra de forma nativa este tipo de datos TEXT mediante una técnica intermedia llamada TOAST, la cual guarda los textos largos en una forma muy eficiente en bloques separadas sin disminuir la velocidad de lectura de las filas principales.

Y en cuanto a la concurrencia el motor utiliza un control multiversión MVCC que garantiza que en caso que varios estudiantes reaccionan o comentan en una misma publicación a la vez, el sistema operativo de la base de datos no sufra bloqueos o se caiga en sus procesos

2. RED SOCIAL PASCUALINA - SCRIPT DDL COMPLETO (PostgreSQL)

```
-- 1. LIMPIEZA AUTOMÁTICA DE SEGURIDAD
DROP TABLE IF EXISTS reaccion CASCADE;
DROP TABLE IF EXISTS conexion CASCADE;
DROP TABLE IF EXISTS asistente_evento CASCADE;
DROP TABLE IF EXISTS miembro_grupo CASCADE;
DROP TABLE IF EXISTS estudiante_habilidad CASCADE;
DROP TABLE IF EXISTS estudiante_interes CASCADE;
DROP TABLE IF EXISTS comentario CASCADE;
DROP TABLE IF EXISTS mentoria CASCADE;
DROP TABLE IF EXISTS evento CASCADE;
DROP TABLE IF EXISTS grupo CASCADE;
DROP TABLE IF EXISTS publicacion CASCADE;
DROP TABLE IF EXISTS perfil CASCADE;
DROP TABLE IF EXISTS estudiante CASCADE;
DROP TABLE IF EXISTS interes CASCADE;
DROP TABLE IF EXISTS habilidad CASCADE;

-- 2. TABLAS CATÁLOGO
CREATE TABLE interes (
    id_interes SERIAL,
    nombre VARCHAR(100) NOT NULL,
    descripcion VARCHAR(300),
    CONSTRAINT pk_interes PRIMARY KEY (id_interes)
);

CREATE TABLE habilidad (
    id_habilidad SERIAL,
    nombre VARCHAR(100) NOT NULL,
    descripcion VARCHAR(300),
    CONSTRAINT pk_habilidad PRIMARY KEY (id_habilidad)
);

-- 3. ENTIDAD NÚCLEO
CREATE TABLE estudiante (
    id_estudiante SERIAL,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(150) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    area_estudio VARCHAR(100),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ultima_actividad TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) DEFAULT 'activo',
    CONSTRAINT pk_estudiante PRIMARY KEY (id_estudiante),
    CONSTRAINT uq_estudiante_correo UNIQUE (correo)
```

```
);

-- 4. ENTIDADES DEPENDIENTES
CREATE TABLE perfil (
    id_perfil SERIAL,
    id_estudiante INT NOT NULL,
    biografia TEXT,
    programa VARCHAR(100),
    semestre INT,
    ciudad VARCHAR(100),
    foto_perfil VARCHAR(255),
    fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_perfil PRIMARY KEY (id_perfil),
    CONSTRAINT fk_perfil_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE publicacion (
    id_publicacion SERIAL,
    id_estudiante INT NOT NULL,
    contenido TEXT NOT NULL,
    tipo VARCHAR(50),
    fecha_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_publicacion PRIMARY KEY (id_publicacion),
    CONSTRAINT fk_publicacion_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE grupo (
    id_grupo SERIAL,
    id_estudiante_creador INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    tipo VARCHAR(30),
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) DEFAULT 'activo',
    CONSTRAINT pk_grupo PRIMARY KEY (id_grupo),
    CONSTRAINT fk_grupo_creador FOREIGN KEY (id_estudiante_creador)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

-- 5. ENTIDADES SEGUNDA OLA
CREATE TABLE evento (
    id_evento SERIAL,
    id_estudiante_organizador INT NOT NULL,
```

```
    id_grupo INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    fecha_inicio TIMESTAMP NOT NULL,
    fecha_fin TIMESTAMP NOT NULL,
    lugar VARCHAR(150),
    modalidad VARCHAR(30),
    estado VARCHAR(20) DEFAULT 'programado',
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_evento PRIMARY KEY (id_evento),
    CONSTRAINT fk_evento_organizador FOREIGN KEY (id_estudiante_organizador)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_evento_grupo FOREIGN KEY (id_grupo)
        REFERENCES grupo (id_grupo)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE comentario (
    id_comentario SERIAL,
    id_publicacion INT NOT NULL,
    id_estudiante INT NOT NULL,
    contenido TEXT NOT NULL,
    fecha_comentario TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_comentario PRIMARY KEY (id_comentario),
    CONSTRAINT fk_comentario_publicacion FOREIGN KEY (id_publicacion)
        REFERENCES publicacion (id_publicacion)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_comentario_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE mentoria (
    id_mentoria SERIAL,
    id_estudiante_mentor INT NOT NULL,
    id_estudiante_aprendiz INT NOT NULL,
    fecha_asignacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(30) DEFAULT 'activa',
    CONSTRAINT pk_mentoria PRIMARY KEY (id_mentoria),
    CONSTRAINT fk_mentoria_mentor FOREIGN KEY (id_estudiante_mentor)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_mentoria_aprendiz FOREIGN KEY (id_estudiante_aprendiz)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
```

```
        ON DELETE CASCADE,
    CONSTRAINT chk_diferente_usuario CHECK (id_estudiante_mentor <>
id_estudiante_aprendiz)
);

-- 6. TABLAS INTERMEDIAS (N:M)
CREATE TABLE estudiante_interes (
    id_estudiante INT NOT NULL,
    id_interes INT NOT NULL,
    CONSTRAINT pk_estudiante_interes PRIMARY KEY (id_estudiante, id_interes),
    CONSTRAINT fk_est_int_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_est_int_interes FOREIGN KEY (id_interes)
        REFERENCES interes (id_interes)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

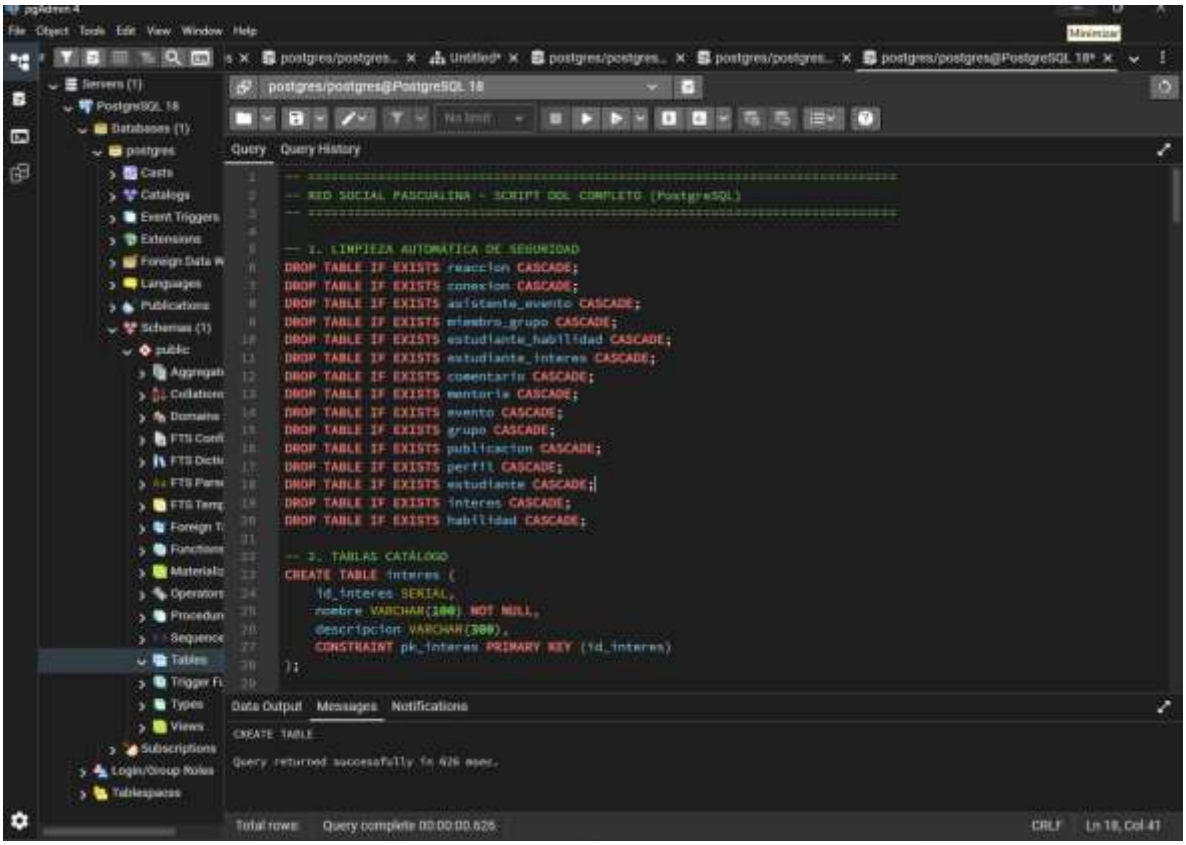
CREATE TABLE estudiante_habilidad (
    id_estudiante INT NOT NULL,
    id_habilidad INT NOT NULL,
    nivel_dominio VARCHAR(50),
    CONSTRAINT pk_estudiante_habilidad PRIMARY KEY (id_estudiante, id_habilidad),
    CONSTRAINT fk_est_hab_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_est_hab_habilidad FOREIGN KEY (id_habilidad)
        REFERENCES habilidad (id_habilidad)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE miembro_grupo (
    id_estudiante INT NOT NULL,
    id_grupo INT NOT NULL,
    rol VARCHAR(50) DEFAULT 'miembro',
    fecha_union TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_miembro_grupo PRIMARY KEY (id_estudiante, id_grupo),
    CONSTRAINT fk_mie_gru_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_mie_gru_grupo FOREIGN KEY (id_grupo)
        REFERENCES grupo (id_grupo)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

```
CREATE TABLE asistente_evento (
    id_estudiante INT NOT NULL,
    id_evento INT NOT NULL,
    estado_asistencia VARCHAR(50) DEFAULT 'confirmado',
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_asistente_evento PRIMARY KEY (id_estudiante, id_evento),
    CONSTRAINT fk_asi_eve_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_asi_eve_evento FOREIGN KEY (id_evento)
        REFERENCES evento (id_evento)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE conexion (
    id_estudiante_origen INT NOT NULL,
    id_estudiante_destino INT NOT NULL,
    estado VARCHAR(30) DEFAULT 'aceptada',
    fecha_conexion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_conexion PRIMARY KEY (id_estudiante_origen,
id_estudiante_destino),
    CONSTRAINT fk_con_origen FOREIGN KEY (id_estudiante_origen)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_con_destino FOREIGN KEY (id_estudiante_destino)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT chk_no_auto_conexion CHECK (id_estudiante_origen <>
id_estudiante_destino)
);

CREATE TABLE reaccion (
    id_estudiante INT NOT NULL,
    id_publicacion INT NOT NULL,
    tipo_reaccion VARCHAR(30) DEFAULT 'like',
    fecha_reaccion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_reaccion PRIMARY KEY (id_estudiante, id_publicacion),
    CONSTRAINT fk_rea_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_rea_publicacion FOREIGN KEY (id_publicacion)
        REFERENCES publicacion (id_publicacion)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```



Implementación y verificación del modelo físico.

El script DDL fue ejecutado en Supabase, utilizando PostgreSQL como sistema gestor de base de datos. La ejecución finalizó correctamente y permitió crear las 15 tablas definidas para la Red Social Estudiantil Pascualina.
Posteriormente, se realizaron consultas de verificación para comprobar la estructura de la base de datos. Se identificaron 21 claves foráneas, 76 campos distribuidos entre las tablas y las restricciones de integridad definidas en el script, incluyendo claves primarias, claves foráneas y restricciones CHECK y NOT NULL.
Esta verificación permitió confirmar que la estructura del modelo físico quedó implementada en el entorno seleccionado.

3. TRANSFORMACIÓN DEL MODELO LÓGICO AL MODELO FÍSICO

La transición del modelo lógico al modelo físico se realizó traduciendo la estructura teórica del diagrama de entidades y relaciones en un script ejecutable de definición de datos (DDL) en el sistema gestor PostgreSQL. Durante este proceso, se aplicaron reglas estrictas de bases de datos para garantizar la integridad, el rendimiento y la seguridad del sistema de la Red Social Pascualina.

- Estandarización de Nombres: Se convirtieron todos los nombres de tablas y atributos de mayúsculas a minúsculas y se aplicó el estilo snake_case (ej. miembro_grupo) para cumplir con las buenas prácticas de PostgreSQL.

- Mapeo de Tipos de Datos: Los identificadores enteros del modelo lógico se transformaron a tipos SERIAL para su autoincremento automático en las llaves primarias. Asimismo, las descripciones abiertas y biografías se mapearon como TEXT, optimizando el espacio mediante el almacenamiento en bloques nativo del motor.

4. INFORME TÉCNICO JUSTIFICANDO LAS DECISIONES

JUSTIFICACION DE ESTRUCTURA DE TABLAS Y TIPOS DE DATOS

Cada columna y tipo de dato en el script DDL se selecciono cuidadosamente para aprovechar al máximo el almacenamiento y así se garantiza el rendimiento del disco duro:

Claves secuenciales (SERIAL) para las claves primarias simples (id_estudiante, id_perfil, id_publicacion, etc.) se elegio por el tipo de serial; por lo tanto, el sistema numera la lista de registros de forma automática entonces el sistema al hacerlo solo, la aplicación tiene menos trabajo y así se garantiza ningún número repetido jamás evitando confusiones o errores entre datos.

Optimización de cadenas (varchar vs texst). Para ahorrar espacio en el disco duro se utilizo limites estrictos en los textos cortos correos a 150 caracteres y nombres a 100 para evitar desperdicio de almacenamiento en el servidor. Se asigno el formato TEXST para que los usuarios escriban párrafos largos y no sufran recortes en sus mensajes en cuanto a la eficiencia el sistema solo consume recursos necesarios para cada tipo de dato

Registro automático de fechas (TIMESTAMP). Configuramos los campos de fecha y hora (como registros, publicaciones y conexiones) para que el sistema los guarde de forma automática. Al usar la propiedad CURRENT_TIMESTAMP, la base de datos registra el momento exacto de cada acción sin que la aplicación tenga que calcularlo. Esto garantiza un control de tiempo preciso, seguro y sin alteraciones, cumpliendo con los estándares requeridos.

5. JUSTIFICACION DEL CICLO DE VIDA DE LOS DATOS (BORRADO Y ACTUALIZACION AUTOMATICA)

Se configuro las reglas de la base de datos así se mantiene la información mas ordenada y limpia sin registros huérfanos o sueltos.

Actualizaciones automáticas (ON UDATE CASCADE). En caso que en un mantenimiento técnico se llegue a cambiar el código o número de identificación del registro principal, ese número cambia automáticamente en todas las tablas relacionadas eso evita realizar cambios manuales uno por uno.

Eliminación en cadena (ON DELETE CASCADE). Se aplicó a los perfiles, publicaciones, comentarios y reacciones. Si un estudiante se retira o elimina su cuenta se borra toda la actividad en la plataforma al mismo tiempo garantizando la privacidad del alumno y a si vez se evita acumular basura digital de datos innecesarios en los servidores de la institución.

6. JUSTIFICACIÓN DEL MAPEO FUNCIONAL FRENTE A LOS REQUERIMIENTOS DEL SISTEMA

REQUERIMIENTO	SOLUCION FISICA
Crear perfiles (Áreas de estudio, intereses y habilidades)	Se diseñó la tabla independiente estudiante para los datos de acceso, conectada mediante una relación de uno a uno (1:1) con la tabla perfil para almacenar datos descriptivos (como la ciudad, biografía y programa académico). Para mapear los intereses y habilidades en disciplinas o deportes sin generar redundancia, se crearon los catálogos maestros interés y habilidad, los cuales se vinculan a los alumnos a través de las tablas intermedias estudiante_interes y estudiante_habilidad (relaciones N:M). En esta última, se incluyó el campo nivel_dominio para calificar la destreza del estudiante.
Conectar compañeros y ofrecer mentoría	Las relaciones entre los mismos alumnos se resolvieron mediante lógica autorreferenciada. Para los seguidores con intereses comunes, se implementó la tabla conexión, validando con un CHECK que un estudiante no se siga a sí mismo. Para el apoyo académico de cursos avanzados, se estructuró la tabla mentoría, vinculando un tutor (id_estudiante_mentor) con

	un alumno (id_estudiante_aprendiz), asegurando la integridad del proceso mediante restricciones de clave foránea en cascada.
publicar actualizaciones (preguntas recursos memes)	Se creó la tabla publicacion para almacenar el cuerpo principal de las actualizaciones usando el tipo de datos TEXT, óptimo para soportar desde preguntas cortas hasta enlaces de recursos tecnológicos. El atributo tipo clasifica si el aporte es un meme, una noticia o una duda. Esta estructura se complementa con la tabla comentario (relación 1: N) para abrir hilos de discusión y la tabla asociativa reacción para cuantificar las interacciones lúdicas ("me gustas") de la comunidad.
Crear y unirse a grupos (estudio, hackatones y clubes)	Se definió la tabla grupo, la cual almacena el nombre del club (ej. videojuegos, programación competitiva) y cuenta con un campo obligatorio (id_estudiante_creador) para identificar al líder administrador. El proceso de unión voluntaria de los estudiantes se gestiona físicamente en la tabla intermedia miembro_grupo, la cual registra la fecha exacta de ingreso (fecha_union) y el rol asignado dentro del equipo de trabajo.
Programar eventos (reuniones, talleres y actividades sociales)	Se implementó la tabla evento, la cual depende jerárquicamente de un estudiante organizador y de un grupo que respalda la actividad. Para garantizar la coordinación exacta, se utilizaron campos de tipo TIMESTAMP NOT NULL para la fecha_inicio y fecha_fin. La asistencia y confirmación de la comunidad estudiantil a estos talleres o reuniones sociales se controla de forma estricta en la tabla asociativa asistente_evento.

7. Diccionario de Datos del Modelo Físico

Tabla 1: estudiante (Datos de Cuenta)

Campo	Tipo	Nulidad	Llave / Restricción	Descripción
id_estudiante	SERIAL	NOT NULL	PK	Código auto incrementable único.
nombre	VARCHAR(100)	NOT NULL	Ninguna	Nombre completo del alumno.
correo	VARCHAR(150)	NOT NULL	UNIQUE	Correo institucional de acceso.
password_hash	VARCHAR(255)	NOT NULL	Ninguna	Contraseña cifrada por seguridad.
area_estudio	VARCHAR(100)	NULL	Ninguna	Facultad o área académica.
fecha_registro	TIMESTAMP	NOT NULL	DEFAULT NOW()	Fecha automática de registro.
ultima_actividad	TIMESTAMP	NOT NULL	DEFAULT NOW()	Última conexión en el sistema.
estado	VARCHAR(20)	NOT NULL	DEFAULT 'activo'	Estado de la cuenta (activo/inactivo).

Tabla 2: perfil (Datos Sociales)

Campo	Tipo	Nulidad	Llave / Restricción	Descripción
id_perfil	SERIAL	NOT NULL	PK	Identificador único del perfil.
id_estudiante	INT	NOT NULL	FK (estudiante)	Código del alumno (Cascada).
biografia	TEXT	NULL	Ninguna	Descripción libre del estudiante.
programa	VARCHAR(100)	NULL	Ninguna	Carrera tecnológica o profesional.
semestre	INT	NULL	Ninguna	Semestre académico actual.
ciudad	VARCHAR(100)	NULL	Ninguna	Ciudad de residencia del alumno.
foto_perfil	VARCHAR(255)	NULL	Ninguna	Ruta o enlace de la imagen.
fecha_actualizacion	TIMESTAMP	NOT NULL	DEFAULT NOW()	Última modificación del perfil.

Tabla 3: publicacion (Muro de Contenidos)

Campo	Tipo	Nulidad	Llave / Restricción	Descripción
id_publicacion	SERIAL	NOT NULL	PK	Código único del post.
id_estudiante	INT	NOT NULL	FK (estudiante)	Autor de la publicación (Cascada).
contenido	TEXT	NOT NULL	Ninguna	Texto, dudas académicas o memes.
tipo	VARCHAR(50)	NULL	Ninguna	Categoría (noticia, meme, duda).
fecha_publicacion	TIMESTAMP	NOT NULL	DEFAULT NOW()	Tiempo automático de publicación.

Tabla 4: grupo (Clubes de Estudio)

Campo	Tipo	Nulidad	Llave / Restricción	Descripción
id_grupo	SERIAL	NOT NULL	PK	Código único del grupo.
id_estudiante_creador	INT	NOT NULL	FK (estudiante)	Alumno líder o administrador.
nombre	VARCHAR(100)	NOT NULL	Ninguna	Nombre del club o semillero.
descripcion	TEXT	NULL	Ninguna	Objetivos o metas del equipo.
tipo	VARCHAR(30)	NULL	Ninguna	Tipo de club (hackatón, hobby).
fecha_creacion	TIMESTAMP	NOT NULL	DEFAULT NOW()	Fecha automática de apertura.
estado	VARCHAR(20)	NOT NULL	DEFAULT 'activo'	Estado operativo del grupo.

Tabla 5: comentario (Hilos de Discusión)

Campo	Tipo	Nulidad	Llave / Restricción	Descripción
id_comentario	SERIAL	NOT NULL	PK	Código único del comentario.
id_publicacion	INT	NOT NULL	FK (publicacion)	Post al que pertenece (Cascada).
id_estudiante	INT	NOT NULL	FK (estudiante)	Alumno que escribe (Cascada).
contenido	TEXT	NOT NULL	Ninguna	Texto con la respuesta o aporte.
fecha_comentario	TIMESTAMP	NOT NULL	DEFAULT NOW()	Tiempo automático del comentario.

Tabla 6: reacción (Me Gustas - Relación N:M)

Campo	Tipo	Nulidad	Llave / Restricción	Descripción
id_estudiante	INT	NOT NULL	PK / FK (estudiante)	Alumno que reacciona (Cascada).
id_publicacion	INT	NOT NULL	PK / FK (publicacion)	Post que recibe el like (Cascada).
tipo_reaccion	VARCHAR(30)	NOT NULL	DEFAULT 'like'	Tipo de interacción lúdica.
fecha_reaccion	TIMESTAMP	NOT NULL	DEFAULT NOW()	Tiempo de la interacción.

Tabla 7: mentoría (Relación Autorreferenciada)

Campo	Tipo	Nulidad	Llave / Restricción	Descripción
id_mentoria	SERIAL	NOT NULL	PK	Código único de la mentoría.
id_estudiante_mentor	INT	NOT NULL	FK (estudiante)	Alumno tutor de cursos avanzados.
id_estudiante_aprendiz	INT	NOT NULL	FK (estudiante)	Alumno guiado.
fecha_asignacion	TIMESTAMP	NOT NULL	DEFAULT NOW()	Fecha de inicio del proceso.
estado	VARCHAR(30)	NOT NULL	DEFAULT 'activa'	Estado de la tutoría (activa/fin).
Restricción Check	`CHECK`	N/A	mentor <> aprendiz	Evita que un alumno sea su propio mentor.

Tabla 8: conexión (Seguidores / Amigos)

Campo	Tipo	Nulidad	Llave / Restricción	Descripción
id_estudiante_origen	INT	NOT NULL	PK / FK (estudiante)	Estudiante que sigue (Cascada).
id_estudiante_destino	INT	NOT NULL	PK / FK (estudiante)	Estudiante seguido (Cascada).
estado	VARCHAR(30)	NOT NULL	DEFAULT 'aceptada'	Estado del vínculo social.
fecha_conexion	TIMESTAMP	NOT NULL	DEFAULT NOW()	Tiempo automático del enlace.
Restricción Check	`CHECK`	N/A	origen <> destino	Evita que un usuario se siga a sí mismo.

Tablas de Catálogos e Intermedias Cortas

Tablas: interés e habilidad

Tabla	Campo	Tipo	Nulidad	Restricción	Descripción
interes	id_interes	SERIAL	NOT NULL	PK	Identificador único del interés.
	nombre	VARCHAR(100)	NOT NULL	Ninguna	Nombre del gusto o hobby.
	descripcion	VARCHAR(300)	NULL	Ninguna	Detalle breve del interés.
habilidad	id_habilidad	SERIAL	NOT NULL	PK	Identificador de la destreza.
	nombre	VARCHAR(100)	NOT NULL	Ninguna	Nombre de la disciplina o deporte.
	descripcion	VARCHAR(300)	NULL	Ninguna	Detalle breve de la habilidad.

Tablas Asociativas N:M restantes (estudiante_interes, estudiante_habilidad, miembro_grupo, asistente_evento)

Tabla	Campo 1 (PK / FK)	Campo 2 (PK / FK)	Campo Atributo	Descripción
estudiante_interes	id_estudiante (INT)	id_interes (INT)	Ninguno	Une alumnos con sus pasiones.
estudiante_habilidad	id_estudiante (INT)	id_habilidad (INT)	nivel_dominio (VARCHAR)	Registra destrezas y su nivel.
miembro_grupo	id_estudiante (INT)	id_grupo (INT)	rol (VARCHAR)	Controla quién se une a cada club.
asistente_evento	id_estudiante (INT)	id_evento (INT)	estado_asistencia	Confirmación a talleres o reuniones.

CONCLUSIONES

SONIA: al realizar esta actividad comprendí de manera practica como influye el diseño en una base de datos en su rendimiento final. Uno de los grandes aprendizajes es el manejo de PostgreSQL y entender de borrar y crear tablas en un orden jerarquice correcto Resolver de manera autónoma los errores de restricciones y relaciones ya existentes me dio una perspectiva real de los retos que enfrenta un administrador de sistemas. Finalmente, logré conectar cómo las necesidades del usuario; como registrar ubicaciones geográficas o gestionar hackatones; se transforman en tablas físicas eficientes e interconectadas.

CAMILA: Durante el desarrollo de esta actividad comprendí la importancia de transformar un modelo lógico en una base de datos física mediante sentencias DDL. Al implementar el script en PostgreSQL utilizando Supabase, pude comprobar cómo se crean las tablas y cómo las claves primarias, las claves foráneas y las restricciones permiten mantener la integridad de la información. Una de las dificultades fue familiarizarme con la herramienta y verificar que las tablas y sus relaciones se hubieran creado correctamente. Esta experiencia me permitió fortalecer mis conocimientos sobre bases de datos y reconocer la importancia de planificar y validar su estructura antes de utilizarla en una aplicación real.

Enlace Video: https://youtu.be/GvceqvH-0jU

BIBLIOGRAFÍA

- Canepa, G. (2016). PostgreSQL Database Handbook. Recuperado de http://dev.cs.ovgu.de/db/PostgreSQL-Database-Handbook.pdf (dev.cs.ovgu.de in Bing)
- Elmasri, R., & Navathe, S. B. (2007). Fundamentos de sistemas de bases de datos (5ª ed., pp. 317-344). Pearson Education.
- Hsu, L., & Obe, R. (2015). Essential PostgreSQL DZone RefCardz. Recuperado de https://ugoproto.github.io/ugodoc/pdf/52492-rc071_postgresql_2.pdf (ugoproto.github.io in Bing)
- Mannino, M. V. (2007). Administración de bases de datos: diseño y desarrollo de aplicaciones (3ª ed.). McGraw-Hill Interamericana.